

Machine Learning: Fundamentos y conceptos relevantes

00. Machine Learning basics

Machine Learning

Programa computacional que aprende de la experiencia respecto a una clase con un desempeño concreto. En otras palabras, es el proceso de resolver un problema real al recopilar un conjunto de datos y construir algorítmicamente un modelo estadístico basado en ese conjunto de datos.

Tipos de ML

1. **Supervised learning:** La información está previamente categorizada (el dataset tienen labels claros).
 - *Classification:* Predecir la clasificación de una etiqueta de un ejemplo sin etiquetar.
 - *Regression:* Predecir una etiqueta de valor real (target) dado un ejemplo sin etiquetar.
2. **Unsupervised learning:** La información no tiene ningún tipo de clasificación (el dataset no tiene labels).
 - *Clustering:* Asignar una etiqueta a los ejemplos utilizando un conjunto de datos sin etiquetar.
 - *Reducción de dimensione:* Elimina características redundantes o altamente correlacionadas; también reduce el ruido en los datos, lo que contribuye a la interpretabilidad del modelo.
 - *Asociación:* Identifica secuencias recurrentes.
3. **Semi-supervised learning:** Es una combinación de ambos. Hay datos categorizados y otros que no.

CRISP-DM

Metodología estándar más utilizada para guiar proyectos de minería de datos y ciencia de datos. Se compone de los siguientes pasos:

 Business →  Data →  Prep →  Model →  Eval →  Deploy → 
(repetir)

1. **Business Understanding** — Entender el objetivo del negocio
2. **Data Understanding** — Explorar y describir los datos
3. **Data Preparation** — Limpiar y transformar los datos
4. **Modeling** — Entrenar y ajustar modelos
5. **Evaluation** — Validar resultados vs. objetivos
6. **Deployment** — Poner el modelo en producción El proceso es iterativo: Se puede regresar a fases anteriores.

Otros conceptos relevantes

- **Parámetros:** Coeficientes internos del modelo que se obtienen automáticamente durante el entrenamiento.
- **Hiperparámetros:** Coeficientes numéricos (externos) elegidos por el programador o analista.
- **StandardScaler:** Estandariza features a media 0 y desviación 1. Esencial para modelos cuya precisión depende de que todas las variables estén en la misma escala.

$$z = \frac{x - \mu}{\sigma}$$

- **LabelEncoder:** Convierte categorías de texto a enteros (0, 1, 2...). Necesario para que los modelos acepten variables categóricas.

```
In [ ]: # StandardScaler

from sklearn.preprocessing import StandardScaler

scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_train = scaler_X.fit_transform(X_train)
X_test = scaler_X.transform(X_test)
y_train_scaled = scaler_y.fit_transform(y_train).ravel()
y_test_scaled = scaler_y.transform(y_test).ravel()

# LabelEncoder
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))
label_encoders[col] = le # guardar para deployment
```

01. Supervised learning

Decision Tree (DT)

Forma predicciones al dividir recursivamente los diferentes atributos de acuerdo a la estructura del árbol.

```
In [ ]: from sklearn.tree import DecisionTreeClassifier
        from sklearn.model_selection import GridSearchCV

X = balanced_df[num_cols].copy()
y = balanced_df['weather'].copy()

# Dividir set de entrenamiento y de testeo (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

"""
HYPERPARAMETER OPTIMIZATION WITH GridSearchCV
"""
# Setting de hiperparámetros para el árbol
param_grid = {
    'max_depth': [3, 5, 10, 25, 30, None], # Define la profundidad máxi
    'min_samples_split': [2, 5, 10, 20], # Cantidad mínima de division
    'min_samples_leaf': [1, 2, 4, 10], # Número mínimo de muestras que
    'max_features': [None, 'sqrt', 'log2'], # Método usado para defini
    'max_leaf_nodes': [None, 10, 20, 50, 100], # Número máximo de hoja
    'criterion': ['gini', 'entropy'] # Algoritmo a utilizar para la to
}

dtree_grid = DecisionTreeClassifier(random_state=42)

grid_search = GridSearchCV(
    dtree_grid,
    param_grid,
    cv=5, # 5-fold cross-validation
    scoring='accuracy',
    n_jobs=1)

# Ejecutar grid search
print("Ejecución de Grid Search para optimización de parámetros...")
print(f"Total de combinaciones de parámetros: {len(param_grid['max_dep
grid_search.fit(X_train, y_train)

# Train optimized model
dtree_optimized = grid_search.best_estimator_
y_pred_optimized = dtree_optimized.predict(X_test)
```

To remember:

- Pruning: Técnica que reduce el tamaño de los árboles de decisión al eliminar

secciones del árbol que no son críticas.

Random Forest

Construye muchos árboles diferentes y combina los resultados.

```
In [ ]: from sklearn.ensemble import RandomForestClassifier
        from sklearn.model_selection import GridSearchCV, train_test_split

X = balanced_df[num_cols].copy()
y = balanced_df['weather'].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Grid de hiperparámetros para Random Forest
param_grid = {
    'n_estimators': [100, 200, 300],          # Número de árboles
    'max_depth': [5, 10, 20, None],           # Profundidad máxima
    'min_samples_split': [2, 5, 10],          # Muestras mínimas pa
    'min_samples_leaf': [1, 2, 4],            # Muestras mínimas en
    'max_features': ['sqrt', 'log2'],          # Features evaluadas
    'criterion': ['gini', 'entropy'],          # Función de impureza
    'bootstrap': [True],                       # Muestreo con reempl
}

rf = RandomForestClassifier(random_state=42, n_jobs=-1)

# Total de combinaciones
total = 1
for v in param_grid.values():
    total *= len(v)

# GridSearchCV
grid_search = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,                                     # 5-fold cross-validation
    scoring='accuracy',                       # Cambia dependiendo el objetivo
    n_jobs=-1,                                # Usa todos los cores
    verbose=2,                                # Progreso en pantalla
)
grid_search.fit(X_train, y_train)

# Modelo optimizado y predicciones
rf_optimized = grid_search.best_estimator_
y_pred = rf_optimized.predict(X_test)
y_proba = rf_optimized.predict_proba(X_test)[:, 1] # Probabilidad cla
```

To remember:

- Divide el dataset para tomar muestras y asignarlas a cada bootstrap con el fin de entrenar cada árbol. Recibe [A, B,C,D, E] y se entrega [A, C, C, E, A].
- Random feature selection: Se seleccionan aleatoriamente variables del dataset para que en cada división del árbol se usen conjuntos aleatorios para favorecer variabilidad.
- Es "democrático": Se evalúan todos los árboles y el resultado final depende de la respuesta mayoritaria.

K-Nearest Neighbor (KNN)

Lazy learning algorithm porque no genera un modelo al entrenarlo, guarda todos los datos y los compara al recibir una observación nueva. No es paramétrico.

```
In [ ]: from sklearn.utils import resample
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# Buscar la clase con más datos en la columna 'weather' y guardar el v
max_count = df['weather'].value_counts().max()

# Separar el DataFrame por clase y sobremuestrear cada grupo
# hasta alcanzar el número de muestras de la clase mayoritaria.
balanced_parts = [
    resample(
        group,
        replace=True,
        n_samples=max_count,
        random_state=42
    )
    for _, group in df.groupby("weather")
]

# Unir todas las clases balanceadas, mezclar las filas
# y restablecer el índice del nuevo DataFrame.
balanced_df = (
    pd.concat(balanced_parts)
    .sample(frac=1, random_state=42)
    .reset_index(drop=True)
)

# Definir nuevos objetivos con base en los cambios realizados
X = balanced_df[num_cols].copy()
y = balanced_df["weather"].copy()
```

```

# Definir features y target
num_cols = balanced_df.select_dtypes(include='number').columns
X = balanced_df[num_cols].copy()
y = balanced_df['weather'].copy()

# Dividir set de entrenamiento y de testeo (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Estandarización de features para KNN
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled_df = pd.DataFrame(X_train_scaled, columns=X.columns)
X_test_scaled_df = pd.DataFrame(X_test_scaled, columns=X.columns)

"""
Búsqueda de la mejor combinación de parámetros
"""

# Setting de hiperparámetros posibles para el árbol
knn = KNeighborsClassifier(
    n_neighbors=5,          # Número de vecinos (k). Valores clásicos: 3
    weights='distance',    # Ponderación por distancia (vecinos más cer
    metric='minkowski',    # Métrica de distancia flexible
    p=2                    # p=2 corresponde a distancia Euclidiana, p=
)

knn.fit(X_train_scaled, y_train)
y_pred = knn.predict(X_test_scaled)

```

To remember:

- Se debe hacer uso de estandarización debido a que este modelo genera predicciones entorno a las distancias, por ende una desviación muy grande entre los valores puede sesgar su precisión.
- Se puede usar para regresión y clasificación.

Regresión Lineal (OLS)

Modelo que ajusta una relación lineal entre variables predictoras y la variable objetivo minimizando la suma de errores al cuadrado. se usa cuando se busca interpretabilidad (coeficientes, p-valores) y se asume linealidad.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon$$

In []: `import statsmodels.api as sm`

```
X = sm.add_constant(data_anno[feature_cols])
y = data_anno["CO2Emissions"]
model = sm.OLS(y, X).fit()
print(model.summary())
```

MLPRegressor (Red Neuronal)

Red neuronal feed-forward con capas densas. Aprende relaciones no lineales mediante retropropagación.

$$h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)})$$

- **Curva de Aprendizaje (Training vs Validation Error)** es un gráfico del error en train y test a lo largo de las épocas. Detecta overfitting y underfitting.

Situación	Train	Test	Interpretación
Buen ajuste	↓	↓ converge	Modelo equilibrado
Overfitting	↓↓	↑ o estancado	Memoriza train
Underfitting	alto	alto	Modelo muy simple

- **Heatmap de Pesos (Red Neuronal)** es una visualización de `coefs_` (pesos sinápticos) de cada capa. Permite interpretar qué variables influyen más y detectar neuronas "muertas".
- **Predicción vs Real (Regplot + Scatter)** es un gráfico que compara `y_predicho` vs `y_real`. La diagonal perfecta (`y = x`) indica predicción ideal.

```
In [ ]: from sklearn.neural_network import MLPRegressor

mlp = MLPRegressor(
    hidden_layer_sizes=(64, 32), # 64 es el tamaño de la primera capa
    activation="relu", # Se usa relu function como función de activaci
    solver="adam", # Gradient descent method
    learning_rate_init=0.001, # Diferencias de epoch que se hacen: a m
    max_iter=1, # Cantidad de iteraciones
    warm_start=True,
    random_state=42,
    shuffle=True)

for epoch in range(300):
    mlp.fit(X_train, y_train_scaled)
```

02. Unsupervised learning

Principal Component Analysis (PCA)

Técnica de reducción de dimensionalidad que transforma un conjunto de variables correlacionadas en un nuevo conjunto de variables no correlacionadas llamadas componentes principales, ordenadas por la cantidad de varianza que explican.

PCA encuentra nuevos ejes (direcciones) donde los datos varían más:

- **PC1**: dirección de **máxima varianza**.
- **PC2**: dirección de **máxima varianza restante**, ortogonal a PC1.
- **PC3, PC4, ...**: siguen el mismo principio, cada una perpendicular a las anteriores.

$$PC_k = w_{k1}X_1 + w_{k2}X_2 + \dots + w_{kn}X_n$$

donde los pesos w (loadings) se eligen para maximizar la varianza explicada y garantizar ortogonalidad entre componentes.

Paso	Descripción
1. Estandarizar	Media 0, desviación 1 (crítico si las escalas difieren)
2. Matriz de covarianza	Calcular cómo varían las variables juntas
3. Eigenvalores y eigenvectores	Eigenvectores = direcciones (PCs); eigenvalores = varianza explicada
4. Ordenar	De mayor a menor eigenvalor
5. Seleccionar componentes	Quedarse con los primeros k que expliquen suficiente varianza
6. Proyectar	Transformar los datos originales al nuevo espacio

```
In [ ]: from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# 1. Estandarizar
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2. Ajustar PCA con todos los componentes
pca = PCA()
X_pca = pca.fit_transform(X_scaled)
```

```

# 3. Varianza explicada
var_exp = pca.explained_variance_ratio_
var_acum = np.cumsum(var_exp)

# 4. Scree plot
fig, axes = plt.subplots(1, 2, figsize=(13, 4))

axes[0].bar(range(1, len(var_exp)+1), var_exp, color='#2E86AB', edgeco
axes[0].set_xlabel('Componente')
axes[0].set_ylabel('Varianza explicada')
axes[0].set_title('Scree Plot')

axes[1].plot(range(1, len(var_acum)+1), var_acum, marker='o', color='#
axes[1].axhline(0.95, linestyle='--', color='gray', label='95%')
axes[1].set_xlabel('Componentes')
axes[1].set_ylabel('Varianza acumulada')
axes[1].set_title('Varianza acumulada')
axes[1].legend()

plt.tight_layout()
plt.show()

# 5. Elegir k componentes (ej. 95% de varianza)
k = np.argmax(var_acum >= 0.95) + 1
print(f"Número óptimo de componentes: {k}")

pca_final = PCA(n_components=k)
X_reduced = pca_final.fit_transform(X_scaled)
print(f"Forma original: {X_scaled.shape} → Forma reducida: {X_reduced.

# 6. Loadings (importancia de cada variable original en cada PC)
loadings = pd.DataFrame(
    pca_final.components_.T,
    columns=[f'PC{i+1}' for i in range(k)],
    index=X.columns
)
print(loadings)

```

03. Model evaluation

Train/test split

De acuerdo a evaluaciones realizadas, se ha determinado que el porcentaje ideal para dividir el dataset es de:

- 70% para training
- 30% para testing

Cross-validation

Métricas

Las métricas nos pueden ayudar a evaluar si el modelo logra hacer predicciones de utilidad.

Métricas para clasificación

Matriz de confusión

Permite evaluar la efectividad de las predicciones del modelo.

	Predicho: POSITIVO	Predicho: NEGATIVO
Real: POSITIVO	✅ Verdadero Positivo (TP)	❌ Falso Negativo (FN)
Real: NEGATIVO	❌ Falso Positivo (FP)	✅ Verdadero Negativo (TN)

Accuracy

Exactitud: % de predicciones correctas sobre el total. Sirve cuando las clases están balanceadas.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

De todo lo que el modelo dijo "positivo", ¿cuánto era realmente positivo? Importa cuando el costo de un FP es alto

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Sensibilidad / Exhaustividad: De todos los positivos reales, ¿cuántos detectó el modelo? Importa cuando el costo de un FN es alto

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score

Media armónica entre Precision y Recall. Útil cuando hay desbalance y se busca equilibrar ambos errores en un solo número.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

ROC

Curva que muestra el trade-off entre TPR (Recall) y FPR para todos los umbrales de decisión. El AUC (área bajo la curva) resume el desempeño: 0.5 = azar, 1.0 = perfecto.

$$\text{TPR} = \frac{TP}{TP + FN} \quad \text{FPR} = \frac{FP}{FP + TN}$$
$$\text{AUC} = \int_0^1 \text{TPR} d(\text{FPR})$$

Métrica	Pregunta que responde	Fórmula	Cuándo usarla
Accuracy	¿Cuánto acertó en total?	(TP+TN)/Total	Clases balanceadas
Precision	¿Cuánto de lo que dije + era +?	TP/(TP+FP)	FP costoso
Recall	¿Cuánto de lo + real detecté?	TP/(TP+FN)	FN costoso
F1	¿Balance Precision/Recall?	2PR/(P+R)	Desbalance
ROC-AUC	¿Qué tan bien separa clases?	Área TPR vs FPR	Comparar modelos / umbral

Métricas regresión

R² (Coeficiente de Determinación)

Proporción de la varianza de **y** explicada por el modelo. Va de 0 a 1 (o negativo si el modelo es peor que la media).

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Valor	Interpretación
1.0	Ajuste perfecto
0.8 – 1.0	Muy bueno
0.5 – 0.8	Moderado
< 0.5	Pobre

Condition Number

Medida de sensibilidad del modelo a pequeños cambios en los datos. Valores altos indican multicolinealidad o mal condicionamiento numérico.

Regla: > 30 es sospechoso; > 1000 indica problemas serios.

MAE (Mean Absolute Error)

Promedio del error absoluto. Robusto a outliers, misma unidad que **y**.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

RMSE (Root Mean Squared Error)

Raíz del error cuadrático medio. Penaliza más los errores grandes que MAE.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

04. Statistical concepts

Correlación

Mide la fuerza y dirección de la relación lineal entre dos variables. Va de -1 a +1.

$$r_{xy} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

Valor	Interpretación
+1	Correlación positiva perfecta
+0.7 a +1	Fuerte positiva
+0.3 a +0.7	Moderada positiva
-0.3 a +0.3	Débil / nula
-0.7 a -0.3	Moderada negativa
-1	Correlación negativa perfecta

Reminder: Correlación $\neq$ Causalidad, dos variables pueden moverse juntas sin que una cause a la otra.

Multicollinearity

Dos o más variables independientes apuntan a una relación (están altamente correlacionadas entre sí), lo que rompe la independencia que asumen muchos

modelos.

Problemas que causa:

- Coeficientes inestables e interpretación ambigua.
- Varianza inflada de los estimadores.
- Dificulta saber qué variable aporta realmente.
- Sesga la importancia en modelos de árboles.

Herramientas de diagnóstico

- **Matriz de correlación:** Herramienta de diagnóstico que permite ver relaciones lineales entre pares de variables. Si $|r| > 0.8$ entre predictores se debe revisar.
- **Scatter matrix:** ¿Cómo es la distribución de los datos? ¿Delata multicollinearity? Permite ver visualmente relaciones no lineales, clusters o patrones que no se ven en los números.
- **Variance inflation factor:** Inflación de la incertidumbre de una variable explicada mediante otras.

$$VIF_i = \frac{1}{1 - R_i^2}$$

donde R_i^2 es el R^2 de regresar la variable i contra todas las demás.

VIF	Interpretación
1	Sin colinealidad
1 – 5	Aceptable
5 – 10	Moderada
> 10	Grave

Soluciones a la multicolinealidad

Estrategia	Descripción
Eliminar una de las variables correlacionadas	Mantener únicamente lo más interpretable
Combinar variables	Promedio, ratio o diferencia entre ellas
PCA	Componentes principales ortogonales entre sí
	Ridge (L2) o Lasso (L1) penalizan

Regularización

coeficientes

VIF iterativo

Eliminar la de mayor VIF una a una

```
In [ ]: # Matriz de correlación
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 8))
sns.heatmap(df[num_cols].corr(), annot=True, cmap='coolwarm', center=0)
plt.title('Matriz de Correlación')
plt.show()

# Scatter matrix
from pandas.plotting import scatter_matrix

scatter_matrix(df[num_cols], figsize=(12, 12), alpha=0.5, diagonal='hi
plt.suptitle('Scatter Matrix')
plt.show()

# Cálculo VIF
from statsmodels.stats.outliers_influence import variance_inflation_fa
from statsmodels.tools.tools import add_constant
import pandas as pd

X_vif = add_constant(df[num_cols])
vif = pd.DataFrame({
    'Variable': X_vif.columns,
    'VIF': [variance_inflation_factor(X_vif.values, i) for i in range(
}).sort_values('VIF', ascending=False)

print(vif)
```

Overfitting/underfitting

- **Overfitting:** El modelo se ajusta en exceso a los datos del entrenamiento y no funciona con otros (falta de generalidad).
- **Underfitting:** El modelo no logró encontrar patrones dentro de los datos, por lo que no puede hacer predicciones.